GT — Game Tracker

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	GT — Game Tracker — Gerenciador de Jogos e Sessões
Gerente de Projeto	Gabriel Grabowski Bosco — estudante responsável
Gerente/Responsável de GCS-DevOps	Gabriel Grabowski Bosco — estudante responsável
Fábrica de Software / Squad	Desenvolvimento individual — Game Tracker
Repositório principal	https://github.com/gabrielgbosco/game-tracker-flutter.git

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Adaptação do template ao projeto Game Tracker, desenvolvido em Flutter/Dart para a N1 — Faixa 8.	Gabriel Grabowski Bosco

Sumário

Sumário 2
1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 4
1.4 Referências 4
1.5 Visão Geral 5
2. Cultura DevOps e Papéis na Gerência de Configuração 5
3. Ferramentas, Ambientes e Infraestrutura 5
3.1 Ferramentas do Pipeline 5
3.2 Ambientes e Infraestrutura 6
3.3 Estratégia de Branching e Versionamento 6
4. Identificação da Configuração 7
4.1 Itens de Configuração 7
4.2 Convenção de Nomenclatura 7
4.3 Estrutura do Repositório 7
5. Integração Contínua (CI) 7
5.1 Etapas do Pipeline de CI 8
5.2 Política de Pull Request e Revisão de Código 8
5.3 Critérios de Qualidade (Quality Gate) 8
6. Entrega e Implantação Contínua (CD) 8
6.1 Etapas do Pipeline de CD 8
6.2 Estratégias de Implantação 8
6.3 Rollback e Recuperação 9
7. Infraestrutura como Código (IaC) 9
7.1 Versionamento de Infraestrutura 9
7.2 Ambientes Efêmeros 9
8. Controle de Configuração e Mudança 9
8.1 Processo de Solicitação de Mudança 9
8.2 Comitê de Controle de Mudança (CCM) 9
9. Baselines e Releases 10
9.1 Geração Automática de Baseline 10
9.2 Notas de Versão (Release Notes) 10
10. Estimativa do Status de Configuração 10
10.1 Monitoramento e Observabilidade 10
10.2 Métricas DORA 10
10.3 Relatórios e Auditorias 10
11. Marcos 10

12. Treinamento e Recursos 11
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 11
14. Quadro-Resumo do Pipeline CI/CD 11

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado ao desenvolvimento individual do aplicativo Game Tracker, desenvolvido em Flutter e Dart para a atividade N1 — Faixa 8. O plano abrange o código-fonte, os modelos de dados, as telas, os widgets reutilizáveis, os recursos visuais, o arquivo de dependências e a documentação do projeto. O aplicativo utiliza dados locais em memória durante a execução e não possui backend, banco de dados, API externa, autenticação ou infraestrutura de produção.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.

1.5 Visão Geral

No Game Tracker, a Gerência de Configuração é aplicada de forma compatível com um projeto acadêmico individual. O controle concentra-se na organização do código-fonte, na identificação dos arquivos e dependências, na validação manual das funcionalidades e na preservação de uma versão estável do projeto para entrega. Não foram identificados pipelines automatizados de CI/CD, ambientes de staging ou produção, IaC ou serviços de nuvem no pacote entregue.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Estudante responsável: define o escopo, implementa as funcionalidades, organiza as entregas e valida os requisitos da atividade.
Engenheiro DevOps / Release Engineer	Não há função DevOps formal no escopo atual. O estudante é responsável pela organização do projeto e pela preparação dos arquivos de entrega.
Desenvolvedor(a)	Implementa as telas, modelos, widgets, navegação, regras de alteração de horas/avaliação e gerenciamento do estado local.
SRE / Operações *Site Reliability Engineering*	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).*
QA / Automação de Testes	Realiza validações manuais das funcionalidades obrigatórias e verifica o comportamento do aplicativo durante a execução.
Segurança (DevSecOps)	Não há pipeline de segurança implementado. As dependências são declaradas no pubspec.yaml e devem ser mantidas compatíveis com o projeto.
Comitê de Controle de Mudanças (CCM)	Não aplicável formalmente ao projeto individual. Mudanças são avaliadas e controladas pelo estudante responsável.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas e recursos identificados no projeto entregue:

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Não identificado no pacote entregue	O código-fonte foi disponibilizado em arquivo ZIP; não foi fornecido endereço de repositório.
Linguagem e framework	Dart + Flutter	Desenvolvimento do aplicativo mobile e composição das telas/widgets.

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Editor/execução	Flutter SDK, IDE e emulador/dispositivo Android	Execução, depuração e validação local do aplicativo.
Gerenciador de dependências	pubspec.yaml / Flutter Pub	Declaração e instalação das dependências do projeto.
Gráficos	fl_chart ^0.68.0	Geração do gráfico de barras de horas jogadas por jogo.
Recursos visuais	Imagens locais em assets/images/	Exibição das capas ilustrativas dos seis jogos.
CI/CD	Não implementado no pacote entregue	As validações são realizadas manualmente durante o desenvolvimento.
Infraestrutura / containers / segredos	Não aplicável	O aplicativo não possui backend, nuvem, containers ou credenciais de serviço.

3.2 Ambientes e Infraestrutura

O projeto não possui ambientes de Desenvolvimento, Homologação ou Produção provisionados por IaC. A execução ocorre localmente em ambiente Flutter configurado pelo estudante, utilizando emulador Android ou dispositivo compatível.

Ambiente	Finalidade	Gatilho de promoção
Execução local	Execução e validação do aplicativo Flutter em emulador Android ou dispositivo compatível.	Execução de flutter run após instalação das dependências.
Homologação / Staging	Não existente no escopo atual.	Não aplicável.
Produção	Não existente no escopo atual; não há publicação remota.	Não aplicável.

3.3 Estratégia de Branching e Versionamento

Recomenda-se estratégia de Trunk-Based Development (ou GitHub Flow), com branches de vida curta, para viabilizar integração contínua real:

- main/trunk — sempre implantável (deployable); protegida por regras de PR obrigatório e pipeline verde.
- feature/<descrição-curta> — branches de curta duração (idealmente < 1-2 dias) para cada mudança.
- release/x.y (opcional) — utilizada apenas se houver necessidade de estabilizar uma versão antes do deploy.
- hotfix/<descrição> — correções emergenciais aplicadas diretamente sobre produção e retro-integradas ao trunk.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor…), permitindo geração automática de changelog e da próxima versão pelo pipeline.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação e dos testes automatizados
- Definições de pipeline como código (ex.: .github/workflows, Jenkinsfile, .gitlab-ci.yml)
- Dockerfiles e manifests de containers
- Scripts de Infraestrutura como Código (Terraform, Ansible, Helm charts, manifests Kubernetes)
- Arquivos de configuração por ambiente (variáveis, feature flags) — exceto chaves secretas, que devem ficar ambientes seguros.
- Scripts de migração de banco de dados (N/A)
- Documentação técnica (README, ADRs — Architecture Decision Records)

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Arquivo Dart	snake_case para arquivos Dart	game_detail_screen.dart
Classe Dart	PascalCase	GameDetailScreen
Variável/método	camelCase	hoursPlayed / addHours()
Versão do projeto	Versão declarada no pubspec.yaml	1.0.0+1
Dependência	Nome e versão no pubspec.yaml	fl_chart: ^0.68.0

4.3 Estrutura do Repositório

O projeto está organizado em um aplicativo Flutter com a seguinte estrutura principal:
• lib/main.dart — ponto de entrada do aplicativo.
• lib/theme/app_theme.dart — cores e estilos centrais.
• lib/models/game.dart — modelo de dados Game, incluindo regras para horas e avaliações.
• lib/data/game_repository.dart — coleção de jogos mantida em memória e cálculos agregados.
• lib/screens/ — telas de Dashboard, lista de jogos, detalhes, estatísticas e recomendação aleatória.
• lib/widgets/ — componentes reutilizáveis GameCard, StatTile e SectionTitle.
• assets/images/ — imagens locais dos seis jogos.
• pubspec.yaml — dependências e configuração dos recursos.
Não existem diretórios de IaC, pipelines ou infraestrutura, pois não fazem parte do escopo do aplicativo.

5. Integração Contínua (CI)

5.1 Etapas do Pipeline de CI

1. Checkout do código e restauração de dependências (com cache).
2. Build da aplicação.
3. Análise estática de código (lint) e verificação de padrões de codificação.
4. Testes unitários, com medição de cobertura mínima.
5. Testes de integração.
6. Análise de segurança do código (SAST) e de dependências (SCA).
7. Empacotamento e publicação do artefato/imagem versionado no registro.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação de outro desenvolvedor/colega antes do merge ("quatro olhos").
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente permitido com pipeline de CI verde (quality gate).

5.3 Critérios de Qualidade (Quality Gate)

Para o escopo atual, o quality gate é baseado em: projeto compilável no Flutter; execução sem erros bloqueantes; dependências declaradas no pubspec.yaml; verificação de lint conforme flutter_lints, quando executada; navegação funcional entre as telas; atualização correta das horas e avaliações; manutenção das horas não negativas; avaliações limitadas entre 0 e 10; funcionamento do gráfico e do sorteio aleatório.

6. Entrega e Implantação Contínua (CD)

Não há pipeline de CD no escopo atual. O aplicativo é executado localmente e não possui processo de promoção entre ambientes ou implantação em produção.

6.1 Etapas do Pipeline de CD

8. Deploy automático em ambiente de Homologação/Staging.
9. Execução de testes automatizados de aceitação/E2E em staging.
10. Gate de aprovação para produção (automático ou aprovação manual, conforme criticidade).
11. Deploy em produção com estratégia definida (ver 6.2).
12. Testes de fumaça (smoke tests) pós-deploy e verificação de saúde (health checks).
13. Geração automática de baseline/tag de release e notas de versão (changelog).

6.2 Estratégias de Implantação

Estratégia	Descrição
Implantação local	Execução do aplicativo em emulador ou dispositivo Android para validação manual.
Rolling update / Blue-Green / Canary	Não aplicável; não existe implantação em produção.
Feature Flags	Não identificadas no projeto entregue.

Estratégia	Descrição

6.3 Rollback e Recuperação

Como não existe implantação em produção, não há processo de rollback automatizado. A recuperação depende da restauração de uma versão anterior do código-fonte ou do pacote do projeto, caso exista uma cópia versionada.

7. Infraestrutura como Código (IaC)

Não há infraestrutura de aplicação no escopo atual. O Game Tracker executa localmente e mantém os dados em memória, sem backend, banco de dados, API externa, serviço em nuvem ou ambiente de produção. Portanto, IaC e pipelines de infraestrutura não se aplicam à versão entregue.

7.1 Versionamento de Infraestrutura

- Não se aplica, pois não existe infraestrutura provisionada como código no projeto entregue.
- O estado da infraestrutura (state file) é armazenado de forma centralizada e versionada, nunca em máquinas locais.

7.2 Ambientes Efêmeros

Não se aplica ao projeto atual, pois não há ambientes de revisão ou implantação remota. A validação é realizada localmente pelo estudante.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina são realizadas pelo estudante diretamente no código-fonte, com organização dos arquivos, revisão manual, execução local do aplicativo e validação das funcionalidades afetadas. Caso seja adotado um repositório Git futuramente, recomenda-se registrar as alterações por commits descritivos e manter uma versão estável do projeto.

8.2 Comitê de Controle de Mudança (CCM)

Não há CCM formal no projeto atual. Mudanças de maior impacto são avaliadas pelo próprio estudante antes da entrega, considerando o funcionamento do aplicativo, a preservação dos requisitos obrigatórios e a ausência de regressões.

9. Baselines e Releases

9.1 Geração Automática de Baseline

No escopo acadêmico atual, a baseline corresponde a uma versão estável do código-fonte e dos recursos necessários para executar o aplicativo. Não há geração automática de tags, imagens ou estado de infraestrutura, pois o projeto não possui pipeline de release ou implantação em produção.

9.2 Notas de Versão (Release Notes)

Não há geração automática de notas de versão no projeto entregue. Alterações relevantes podem ser registradas manualmente na documentação ou no histórico do controle de versão, caso este seja adotado.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

O aplicativo não possui ambientes de Homologação e Produção nem monitoramento remoto. A verificação ocorre durante a execução local, observando erros de compilação, comportamento das telas, navegação, atualização do estado e funcionamento das funcionalidades obrigatórias.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

Marcos aplicáveis ao projeto:
- Organização inicial do projeto Flutter e configuração do pubspec.yaml.
- Criação do modelo Game e da coleção inicial com seis jogos.
- Implementação das telas de Dashboard, lista, detalhes, estatísticas e recomendação aleatória.
- Implementação do gerenciamento de estado local em memória.
- Inclusão do pacote fl_chart para o gráfico de horas jogadas por jogo.
- Validação manual do funcionamento do aplicativo e preparação dos arquivos exigidos para a entrega.

12. Treinamento e Recursos

O projeto é desenvolvido individualmente. Os recursos necessários são Flutter SDK, Dart, editor/IDE compatível, emulador ou dispositivo Android e acesso ao terminal para execução de flutter pub get e flutter run.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- Análise automática de vulnerabilidades em dependências (SCA) integrada ao pipeline de CI.
- Atualizações de dependências propostas automaticamente (ex.: Dependabot/Renovate) e submetidas ao mesmo fluxo de PR.

Não há software desenvolvido por subcontratados ou fornecedores externos identificado no projeto. O único pacote externo relevante declarado é o fl_chart, utilizado para gerar o gráfico de barras na tela de Estatísticas. A dependência é declarada no pubspec.yaml e deve ser instalada por meio do gerenciador de dependências do Flutter. As imagens utilizadas são recursos locais gerados para o projeto.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
Validação local	Instalação das dependências, execução do aplicativo e verificação manual das funcionalidades.	Comando flutter run / execução no emulador.
Verificação de código	Análise manual e, quando executado, lint do Flutter.	Execução local de flutter analyze.
Testes funcionais manuais	Verificação de Dashboard, lista, detalhes, alteração de horas/avaliação, estatísticas e sorteio.	Após alterações no código.
Entrega	Compactação do projeto Flutter e preparação da documentação e do vídeo exigidos.	Conclusão da validação da Faixa 8.